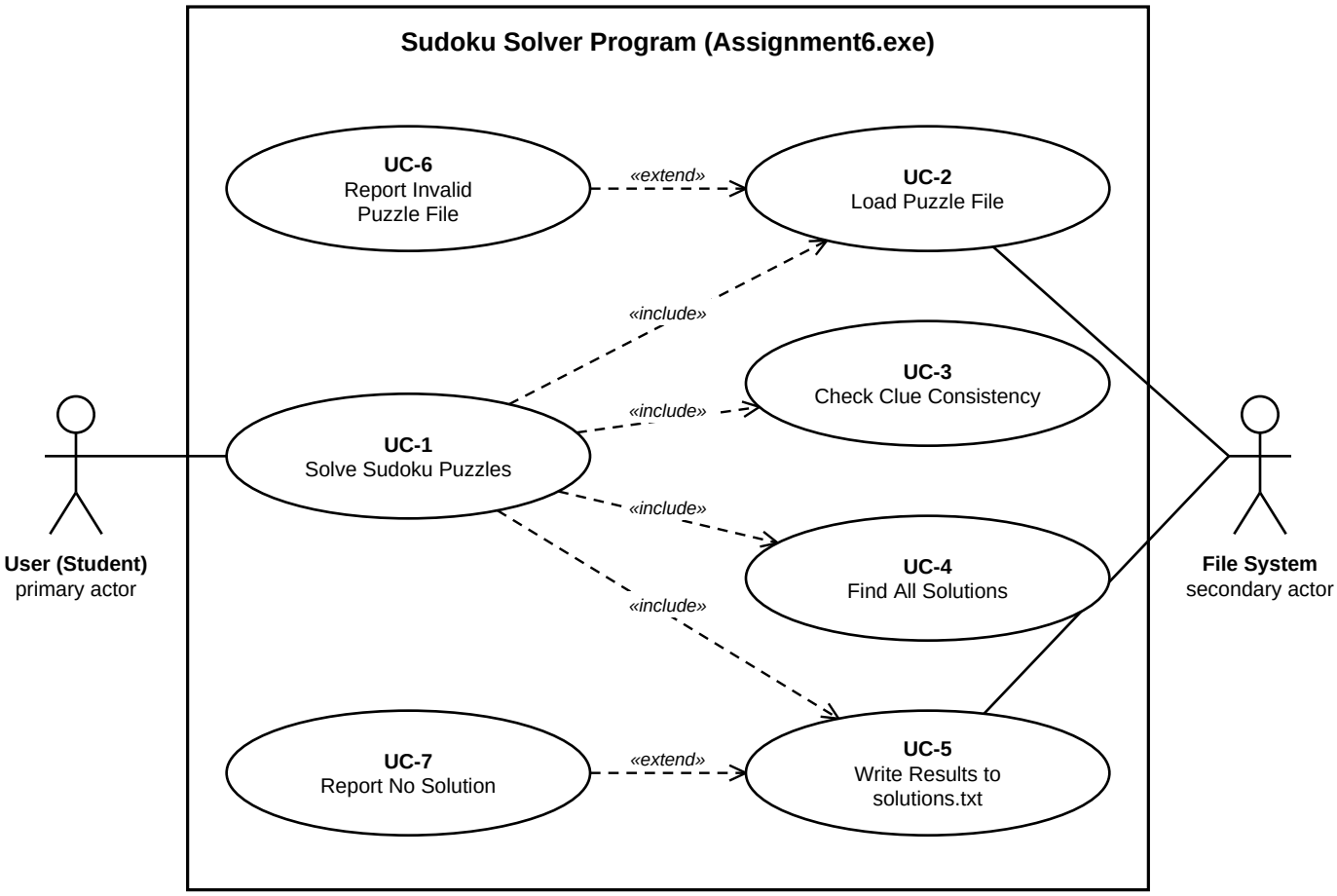
Assignment 6 - UML Use Case Diagram

EECS 348 Software Engineering I (Fall 2026) | Josselyn Bui | Program: Sudoku Solver (Assignment6.cpp) | November 12, 2026
Diagram created with GenAI assistance (Claude, Anthropic) and reviewed by the author.



Solid line: association between an actor and a use case.

Dashed arrow labeled «include»: the base use case always uses the included use case (arrow points to the included one).

Dashed arrow labeled «extend»: optional behavior added to the base use case when its condition holds (arrow points to the base).

Level 1 - Identified Use Cases

ID	Use case	Kind	Actors
UC-1	Solve Sudoku Puzzles	Primary use case	User (primary), File System (secondary)
UC-2	Load Puzzle File	Included in UC-1	File System
UC-3	Check Clue Consistency	Included in UC-1	None (internal)
UC-4	Find All Solutions	Included in UC-1	None (internal)
UC-5	Write Results to solutions.txt	Included in UC-1	File System
UC-6	Report Invalid Puzzle File	Extends UC-2	User (reads console message)
UC-7	Report No Solution	Extends UC-5	None (writes to output file)

Level 2 - Brief Descriptions

ID	Brief description
UC-1	The user runs the program, which reads puzzle1.txt through puzzle5.txt, finds every solution of each puzzle with recursive backtracking, and writes all results to one file, solutions.txt.
UC-2	Reads one puzzle file into a 9x9 board, accepting exactly 81 tokens (digits 1 to 9 or an underscore for a blank).
UC-3	Checks that the given clues contain no duplicate digit in any row, column, or 3x3 box, and records the used digits in bitmasks.
UC-4	Searches the blank cells recursively, trying only legal digits, and records every complete board as a solution.
UC-5	Writes each solution grid and the final solution count (or the no-solution message) to solutions.txt.
UC-6	When a puzzle file is missing, unreadable, or malformed, shows an error on the console, writes "No solution found." for that puzzle, and continues with the next one.
UC-7	When the clues conflict or the search finds zero solutions, writes the single line "No solution found.".

Level 3 - Step-by-Step Flow of Events (UC-1 Solve Sudoku Puzzles)

Main flow

1. The user runs `./Assignment6.exe` in the folder that contains puzzle1.txt to puzzle5.txt.
2. The system creates the output file solutions.txt (alternate flow E1 if it cannot).
3. For each puzzle file, in order from puzzle1.txt to puzzle5.txt, the system repeats steps 4 to 8.
4. The system writes the header line `Puzzle: <file name>` to solutions.txt.
5. The system loads the puzzle file (include UC-2; alternate flow A1 if the file is invalid).
6. The system checks that the clues are consistent (include UC-3; alternate flow A2 if they conflict).
7. The system finds all solutions with recursive backtracking, writing each completed grid to solutions.txt as it is found (include UC-4 and UC-5).
8. The system writes `Solutions found: n` (include UC-5; alternate flow A3 if n is 0).
9. After the last puzzle, the system closes solutions.txt and the program ends with exit code 0.

Alternate and exception flows

A1. Invalid puzzle file (extend UC-6): at step 5 the file is missing, unreadable, has fewer or more than 81 tokens, or has a token that is not 1 to 9 or an underscore. The system prints an error naming the file, writes `No solution found.`, and continues with the next puzzle at step 4.

A2. Conflicting clues (extend UC-7): at step 6 a digit repeats in a row, column, or box. The system writes `No solution found.` and continues with the next puzzle.

A3. No solutions (extend UC-7): at step 7 the search finishes with zero solutions. The system writes `No solution found.` instead of a count and continues with the next puzzle.

A4. Multiple solutions: at step 7 the search does not stop at the first solution. Every solution is written, in the order the search finds them (cells in row-major order, digits in ascending order).

E1. Output file cannot be created (step 2): the system prints an error and the program ends with exit code 1.

E2. Write failure during a puzzle (steps 7 and 8): the system prints an error and continues with the next puzzle.

Level 4 - Detailed Description

UC-1 Solve Sudoku Puzzles

Use case name / ID	Solve Sudoku Puzzles / UC-1
Primary actor	User (the student who runs the program)
Secondary actor	File System (supplies the puzzle files and stores solutions.txt)
Goal	Produce, in one output file, every solution of each of the five puzzles.
Preconditions	1. Assignment6.exe has been compiled with g++ -std=c++17 on the Cycle server. 2. puzzle1.txt to puzzle5.txt are in the folder where the program is run. 3. The user may create files in that folder.
Trigger	The user runs the executable.
Main flow	Steps 1 to 9 in Level 3.
Alternate flows	A1 invalid puzzle file, A2 conflicting clues, A3 no solutions, A4 multiple solutions (Level 3).
Exception flows	E1 output file cannot be created, E2 write failure (Level 3).
Postconditions (success)	solutions.txt exists and holds one section per puzzle, in order. Each section starts with `Puzzle: <file name>`, then either all solution grids followed by `Solutions found: n`, or the single line `No solution found.`. The program ends with exit code 0.
Postconditions (failure)	If solutions.txt cannot be created, no output file exists and the program ends with exit code 1. A bad individual puzzle never stops the run: its section reads `No solution found.` and the rest are still processed.
Business rules	BR1. Solutions use the same format as the input files: nine rows of nine digits separated by single spaces. BR2. Every solution is reported; the search never stops at the first one. BR3. Cells are filled in row-major order and digits tried in ascending order, so the output order is deterministic. BR4. The solution count is kept in a `long long` so very large counts cannot overflow.
Special requirements	Single source file Assignment6.cpp; recursive and object-oriented design (class SudokuSolver); graded on the EECS Cycle servers.
Frequency of use	Once per program run.
Assumptions	Each puzzle file contains exactly 81 whitespace-separated tokens, each a digit 1 to 9 or an underscore.

Included and extending use cases

UC-2 Load Puzzle File (included in UC-1)

Preconditions	The puzzle file name is known (puzzle1.txt to puzzle5.txt).
Flow	1. Open the file. 2. Reset the board, the bitmasks, and the solution counter. 3. Read 81 tokens; an underscore becomes an empty cell and a digit 1 to 9 becomes a clue. 4. Confirm there are no extra tokens and no read error.
Postconditions	Success: the board holds the clues. Failure: the load reports failure and UC-6 runs.

UC-3 Check Clue Consistency (included in UC-1)

Preconditions	A puzzle has been loaded.
Flow	1. Clear the row, column, and box bitmasks. 2. For each clue, test whether its digit is already used in its row, column, or box. 3. If not, record the digit in the three masks; if so, stop with a conflict.
Postconditions	Success: the masks reflect every clue. Failure: the puzzle has no solution and UC-7 runs.

UC-4 Find All Solutions (included in UC-1)

Preconditions	The clues are consistent and the masks are built.
Flow	1. List every blank cell once, in row-major order. 2. For the next blank, compute the legal digits from the three masks. 3. For each legal digit in ascending order: place it, continue with the next blank, then remove it (backtrack). 4. When no blanks remain, add one to the solution count and write the grid (UC-5).
Postconditions	The solution count is final and every solution has been written.

UC-5 Write Results to solutions.txt (included in UC-1)

Preconditions	solutions.txt is open for writing.
Flow	1. Write each completed grid as nine lines of space-separated cells. 2. After the search, write `Solutions found: n` when n is greater than 0.
Postconditions	The puzzle's section of solutions.txt is complete. When n is 0, UC-7 runs instead of step 2.

UC-6 Report Invalid Puzzle File (extends UC-2)

Extension condition	The file is missing, unreadable, or malformed.
Flow	1. Print an error on the console naming the file. 2. Write `No solution found.` to solutions.txt. 3. Continue with the next puzzle.
Postconditions	The puzzle's section reads `Puzzle: <file name>` then `No solution found.`.

UC-7 Report No Solution (extends UC-5)

Extension condition	The clues conflict, or the search finds zero solutions.
Flow	Write the single line `No solution found.` in place of any grids or count.
Postconditions	The puzzle's section is complete and processing continues with the next puzzle.